

Structuri de date

Tema 1

Condiții necesare pentru acceptarea temei:

1. Este obligatorie utilizarea alocării dinamice a memoriei pentru vectori și matrice sau a containerului `std::vector` pentru rezolvarea problemelor, NU se vor puncta rezolvările conțin alocarea statică de memorie. Folosiți sintaxa C++.
2. NU sunt permise variabile globale.
3. SE CERE separarea citirii datelor de rezolvarea problemei! Citiți matricile din fișier!
4. Rezolvați la alegere cel puțin 5 probleme. Fiecare problemă are alături punctajul aferent.
5. Se acordă un punct din oficiu.
6. Se va încarca un singur fișier pdf, salvat cu numele studentului și care se va conține codul din fisiere .cpp/.h. În acest fișier vor fi trecute cerințele, numărul de puncte al problemelor și rezolvările acestora.

Cerințe:

1. Se consideră un vector V cu nr elemente numere naturale. Să se construiască un al doilea vector W , care conține fiecare element din V , care are în componență doar cifre impare, dublicându-le.(1p)
Exemplu: $V = [1, 2, 5, 13, 27, 3, 57] \rightarrow W = [1, 1, 5, 5, 13, 13, 3, 3, 57, 57]$
2. Se consideră un vector de n puncte. Fiecare punct este un element de tipul unei structuri cu două câmpuri, reprezentând coordonatele spațiale (x, y) . Să se afișeze perechile de 4 puncte care pot forma un dreptunghi. Dacă nu există astfel de pereche se va afișa un mesaj corespunzător(1,5p)

3. **Parantezare corectă:** Se dă un șir de paranteze deschise și închise de tip (,), [,], {, }. Să se verifice dacă șirul este corect. Folosiți o stivă (std::stack) pentru rezolvare. **Exemplu:** șirul [()()] este corect, șirul ([]) nu este corect, șirul ()) (nu este corect. (1,5p)
4. Se consideră o structură *fracție*, cu câmpurile de tip *int* *numarator* și *numitor*. Această structură dispune de o metodă *reductie*, care reduce fracția (ex: 12/30 devine 2/5), de funcții pentru operații aritmetice, de funcții de comparare. De asemenea o funcție de transformare în număr zecimal (ex: 2/5 = 0.4). Să se citească un vector de fracții. Fiecare dintre acestea să se reducă. Să se sorteze vectorul cât mai eficient și să se calculeze suma elementelor sale. Scrieți funcții de citire/ afișare pentru fracții. (2p)
5. Se consideră un teren dreptunghiular reprezentat de o matrice de dimensiuni *height* × *width*. Pe acest teren pasc văcuțe și se plimbă găște. Fiecare poziție de pe teren este marcată cu 0 sau 2. Dacă este marcată cu 2, înseamnă că acolo se află două picoare. Văcuțele sunt reprezentate de două perechi picioare aflate pe poziții alăturate (pe verticală sau pe orizontală). Găștele sunt reprezentate de poziții marcate cu 2, înconjurate de 0. Să se afișeze câte văcuțe și câte găște sunt pe teren. (1p)
6. Se consideră un vector de intervale. Se va folosi tipul *pair* pentru a reține cele 2 capete (numere întregi) ale unui interval. Să se definească o funcție care să determine în cel mai eficient mod (utilizând căutare binară) intervalul de lungime minimă în care se află un element dat ca parametru. Dacă acesta nu se află în interiorul niciunui interval, se va afișa un mesaj corespunzător. Exemplu: Dacă se citesc intervalele: [2, 14], [7, 9], [20, 25], [4, 11], [30, 35], pentru valoarea 8 se va afișa intervalul [7, 9]. (1,5p)
7. Să se simuleze o stivă utilizând două cozi. Se va crea structura stivă cu funcțiile de push(), pop(), isEmpty() și clear(), iar cele 2 cozi vor fi câmpuri ale structurii. Puteți folosi implementare proprie de coadă sau queue din C++. (1p)
8. Să se implementeze o listă dublu înlănțuită cu funcționalitățile descrise în continuare. Se cere utilizarea unei structuri *node* care are trei câmpuri: un câmp pentru informație (de tip *int*) și două câmpuri de tip pointer la *node* pentru legăturile către elementele precedent și următor. Se cere utilizarea unei structuri *List* care are ca membri două variabile de tip *node** reprezentând primul respectiv ultimul element din listă, o variabilă de tip *int* reprezentând numărul de elemente din listă și funcțiile:
 - *push_front(key)* - adaugă cheia *key* în capul listei
 - *push_back(key)* - adaugă cheia *key* la finalul listei

- *pop_front()* - șterge primul element din listă
- *pop_back()* - șterge ultimul element din listă
- *find(key)* - caută o cheie *key* în listă - returnează pointer la nodul cu cheia *key* sau nullptr
- *erase(node*Nod)* - șterge un element *Nod* din listă (NU implică căutare). Nodul transmis ca parametru a fost în prealabil găsit cu *find* sau identificat prin parcurgerea listei.
- *remove(int key)* - șterge toate aparițiile cheii *key* (implică parcurgerea listei)
- *insert_after(key1, key2)* - inserează un element cu cheia *key2* după ultima apariție a cheii *key1*. Dacă *key1* nu apare în listă, atunci *key2* se adaugă la finalul listei;
- *empty()* - verifică dacă lista e vidă
- *clear()* - golește lista.
- funcția *print()* - afișează elementele listei
- funcția *size()* - returnează numărul de elemente din listă.

De asemenea să se implementeze următoarele funcții, care nu fac parte din structură:

- *palindrom(List L)* - verifică dacă lista este palindrom
- *compare(List L1, List L2)* - returnează 1 dacă *L1* și *L2* sunt identice și 0 altfel.

în funcția *main* realizați un meniu cu ajutorul unei instrucțiuni *switch*, prin care se oferă opțiuni, corespunzătoare fiecărei funcționalități, precum și o opțiune de EXIT. într-o instrucțiune *while*, se citesc și se execută opțiuni până la alegerea opțiunii de EXIT.

ATENȚIE: Nici o funcție nu trebuie să dea eroare de execuție, dacă se apelează pe o listă vidă!!!(2p)

9. **Implementare coadă.** Să se implementeze o coadă utilizând liste înlănțuite. Vă trebuie:

- o structură *node* cu două câmpuri - un câmp pentru informație (de tipul cerut de problema curentă) și un câmp de tip pointer la *node* pentru legătura la elementul următor.
- o structură *Queue* cu

- două câmpuri de tip pointer la nod, pentru primul și ultimul element inițializate cu NULL (nullptr).
- un câmp *nr_elem* de tip int - numărul de elemente din coada.
- funcția *push(elem)* - pune *elem* la sfarsitul cozii
- funcția *pop()* - elimină elementul de la începutul cozii
- funcțiile *front()* și *back()* returnează primul respectiv ultimul element din coadă
- funcția *empty()* - verifică dacă coada este vidă.
- funcția *clear()* - golește coada
- funcția *size()* - returnează numărul de elemente din coada

Această coadă va fi utilizată în următoarea problemă:

La un examen se pot prezenta candidați pe durata a două zile. În fiecare zi timpul alocat pentru examinare este de t ore ($t \leq 5$). La examen se înscriu n candidați. Se citesc din fișier t , n precum și candidații cu numele (de tip **std::string**). Ei vor fi introduși într-o coadă, de unde vor fi extrași pe rând pentru examinare. Pentru fiecare candidat, care este la rând, se generează o durată aleatorie cu o valoare între 15 minute și 25 minute. În momentul în care timpul t s-a terminat, deci se încheie prima zi de evaluare, candidații care au rămas în coada vor fi extrași pe rând și trecuți într-un fișier de ieșire, care va reprezenta lista candidaților pentru ziua a doua de examinare. (2p)

10. **Evaluarea expresiilor aritmetice.** Se citește dintr-un fișier un șir de expresii aritmetice alcătuite din numere întregi fără semn, operatorii aritmetici +, −, *, /, ^ (reprezentând ridicare la putere) și paranteze rotunde. Expresiile pot fi despărțite între ele prin ; sau prin trecere la rând nou. **Atenție:** algoritmul trebuie să funcționeze corect și dacă există caractere "albe" în expresiile considerate (spații, tab-uri) și dacă NU există! Spațiile albe se ignoră la analizarea acestor expresii.(3p)

Pentru fiecare expresie:

- Să se afișeze întâi expresia aritmetică.
- Să se construiască forma poloneză postfixată și să se afișeze.
- Să se evalueze expresia și să se afișeze rezultatul.
- Să se semnaleze erori în expresie (de parantezare, de operatori, de caractere nepermise). Dacă într-o expresie se găsesc erori se întrerupe analiza/evaluarea acesteia după mesajul de eroare și se trece la următoarea expresie.

Utilizați stive din STL. Folosiți funcții separate pentru construirea formei poloneze pentru o expresie aritmetică stocată într-un **string** și pentru evaluarea pornind de la o formă poloneză.

Exemple:

- $2 + + + 3$ nu este o expresie corectă. Programul semnalează că există prea mulți operatori.
- $2 * ((3 + 4)$ nu este o expresie corectă. Programul semnalează faptul că parantezarea nu este corectă.
Observație: chiar dacă numărul de paranteze deschise este egal cu cel de paranteze închise, parantezarea poate să nu fie corectă. De exemplu $(((($ sau $(((($.
- $23\# + a$ nu este o expresie corectă. Programul semnalează faptul că apar caractere nepermise $\#$, a .
- $3 * 4 - 3 * (24 - 12) - 7$. Programul afișază valoarea -31 . Forma poloneză postfixată este $34 * 32412 - * - 7-$

Algoritmul este descris în documentația de pe e-learning.

11. **Eliminarea parantezelor redundante.** Se citește dintr-un fișier un șir de caractere reprezentând o expresie aritmetică formată din numere, operatori ($+$, $-$, $/$, $*$), litere (reprezentând variabile), paranteze rotunde, eventual spații albe. Se cere eliminarea parantezelor redundante, utilizând o adaptare a algoritmului de la forma poloneză. (5p)

Exemple:

- (a) Pentru expresia: $((x + y) * 3 + ((z)))$ rezultatul algoritmului ar trebui să fie: $((x + y) * 3 + z)$.
- (b) Pentru expresia: $((x + y)) * 3 + ((z - x))$ rezultatul algoritmului ar trebui să fie: $((x + y) * 3 + (z - x))$ sau mai bine $(x + y) * 3 + (z - x)$

12. **Deque.** Implementați o structură de tip *deque*. Această structură trebuie să dispună de un container secvențial în care se vor stoca elementele, de funcții de tip *push-back*, *push-front*, *pop-back*, *pop-front*, *size*, *empty*, toate în complexitate constantă. De asemenea trebuie să permită accesul prin poziție la elemente (de exemplu printr-o funcție *at*) și funcții de inserare / ștergere de pe o anumită poziție.

Pentru implementare folosiți un vector de array-uri (blocuri de memorie de dimensiune fixă). Pentru acest lucru consultați materialul de curs sau puneți întrebări suplimentare la laborator. (3p)